

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
FACULTAD DE INGENIERÍA
INGENIERÍA EN COMPUTACIÓN

Proyecto:

**Simulador del Sistema de Archivos
FiUnamFS con Interfaz de Terminal
FUSE**

Alumnos:

- Bello Sánchez Santiago Arath
- López Romero David Baruc

Materia: Sistemas Operativos

Grupo: 7

Fecha de entrega: 21 de mayo 2026

1. Objetivo del Proyecto

- Desarrollar un simulador interactivo para el sistema de archivos de la Facultad de Ingeniería (FiUnamFS), capaz de realizar operaciones fundamentales de manipulación de almacenamiento virtual a nivel binario.
- Implementar una interfaz de línea de comandos inspirada en el comportamiento de un sistema de archivos en espacio de usuario (FUSE) que permita listar, transferir y eliminar archivos del cluster simulado de manera concurrente y segura.

2. Primer Acercamiento (Análisis del Problema)

- **Simulación y Entorno:** El proyecto trabaja directamente manipulando un archivo binario de almacenamiento emulado que representa el disco rígido virtual bajo el formato FiUnamFS.

Para poder realizar pruebas y validación del sistema, el entorno ya contiene inicialmente un conjunto de archivos preinstalados con propósitos en específico:

- `README.ORG`: Este archivo contiene todas las especificaciones y requerimientos técnicos del diseño del proyecto.
- `logo.png`: Logotipo de la Facultad de Ingeniería.



- `mensaje.jpg`: Imagen emblemática de un gato para el software libre.



- **Cálculos y Especificaciones Técnicas:** Dentro del sistema, cada bloque lógico o unidad de asignación se encuentran determinados por el tamaño del cluster, el cual es establecido por el software:

Tamaño del Clúster = 2048 bytes

Las posiciones físicas dentro del disco (*byte offsets*) para la manipulación y lectura directa de datos estructurados se van a resolver de manera dinámica mediante la multiplicación del índice del cluster por la constante dimensional:

$$\text{Posición Inicial (Bytes)} = \text{initial_cluster} \times 2048$$

- **Restricciones del Sistema:** Las operaciones de persistencia e indexación se encuentran acotadas por los límites del directorio raíz mapeado internamente. Remove de manera física los registros no reubica el almacenamiento inmediatamente, sino que aplica máscaras binarias de invalidación sobre descriptores fijos.

3. Solución Propuesta

La solución propuesta se divide en dos partes, la primera se encarga de la traducción binaria y la segunda es un bucle de eventos interactivo para cumplir las operaciones estándar del shell UNIX:

- **Validación de Datos de Entrada:** El software implementa una rutina de parseo cronológico que recibe cadenas crudas de metadatos del sistema de archivos (YYYYMMDDHHMMSS) y las convierte mediante desgloses indexados en un formato de estampa temporal estandarizado ISO (YYYY-MM-DD HH:MM:SS).
- **Operación de Lectura / Listado (ls):** Invoca el procedimiento interno de mapeo del directorio, itera sobre la colección dinámica `lista_archivos` y renderiza en la terminal virtual el binomio de atributos conformado por el identificador nominal del archivo (`name`) junto con su peso exacto en bytes (`size`).
- **Operación de Escritura / Inserción (cp):** Divide el flujo en dos direcciones complementarias: `copia_T0_MyPC` extrae la secuencia de bytes crudos desde los sectores lógicos asignados en el disco virtual hacia el sistema operativo anfitrión, mientras que `copia_T0_FiUnamFS` realiza el empaquetado inverso localizando bloques vacíos disponibles.
- **Operación de Eliminación (rm):** Traduce el nombre del archivo a una dirección física binaria utilizando una estructura asociativa de metadatos. El programa se posiciona en el descriptor y escribe secuencialmente la firma de borrado físico del sistema consistente en los bytes lógicos `b'/' + b'#' * 14`, liberando las llaves en las tablas dinámicas en memoria.

4. Entorno y Dependencias

- **Lenguaje de Programación Utilizado:** Python 3. Se seleccionó por su eficiencia en el manejo de estructuras de datos, su tipado dinámico para la interacción con descriptores virtuales y su capacidad para controlar directamente punteros en archivos binarios.
- **Módulos y Librerías Nativas:**
 - `os` y `sys`: Para interactuar con el entorno de ejecución, administrar buffers y capturar de argumentos de arranque por consola.

- **struct:** Desempaquetado y traducción de estructuras binarias de C/C++ en tipos lógicos de Python.
 - **threading:** Para la gestión de la concurrencia y aislamiento en la ejecución de hilos.
 - **math y datetime:** Para realizar cálculos sobre la geometría de los bloques y dar formato a las marcas de tiempo del sistema de archivos.
- **Consideraciones de Ejecución y Configuración:** El sistema consta de dos módulos principales. El punto de entrada se localiza en `FUSE.py`. Para inicializar la consola interactiva se requiere pasar como parámetro obligatorio la ruta física de la imagen de almacenamiento en disco mediante el comando:

```
python FUSE.py <ruta_sistemaArchivos>
```

5. Implementación (Arquitectura de Software)

La implementación está realizada bajo la paradigma de la Programación Orientada a Objetos (POO) y emplea una arquitectura modular:

- **Clase File:** Modela y encapsula los metadatos de un archivo individual. En ella se administran propiedades como tamaño de clúster, nombre del archivo, tamaño físico, coordenadas lógicas de inicio y flujos de lectura de contenido binario mediante métodos como `obtener_contenido()`.
- **Clase FiUnamFS:** Es el núcleo central que orquesta las operaciones de bajo nivel del simulador. Almacena en memoria diccionarios clave-valor como `mapDirectorio` y `archivos_validos`, lo cual, permite la mutabilidad del almacenamiento virtual y facilita la ejecución de rutinas como `eliminar_archivo()`.

```

1 def eliminar_archivo(self, nameFile: str) -> bool:
2     if nameFile not in self.archivos_validos:
3         print("No existe archivo con nombre:", nameFile)
4         return False
5
6     byte_inicio = self.mapDirectorio.get(nameFile)
7     try:
8         with open(self.path, 'rb+') as file:
9             file.seek(byte_inicio)
10            marca_borrado = b'/' + b'#' * 14
11            file.write(marca_borrado)
12
13            del self.archivos_validos[nameFile]
14            del self.mapDirectorio[nameFile]
15            self.lista_archivos = [f for f in self.lista_archivos
16 if f.name != nameFile]
17            print(f"Archivo '{nameFile}' eliminado exitosamente.")
18            return True
19        except Exception as e:
20            print(f"Error inesperado al escribir en el disco: {e}")
21            return False

```

Listing 1: Eliminación binaria sobrescribiendo metadatos físicos

- **Módulo de Consola (FUSE.py):** Implementa el ciclo de ejecución principal a través de la función `iniciar_consola(ruta_imagen)`, este módulo procesa el flujo de entrada de la terminal virtual, analizando los comandos introducidos por el usuario como (`ls`, `cp`, `rm`, `exit`) y realiza las instrucciones correspondientes al gestor del sistema.

6. Descripción de la Concurrency y Sincronización Utilizada

- **Ejecución multihilo:** La solución emplea la librería nativa threading para gestionar la concurrencia. El diseño de la terminal interactiva tipo FUSE delega las operaciones críticas de entrada y salida (I/O) a hilos de ejecución secundarios, asegurando de esta manera que los procesos independientes operen en paralelo sin bloquear el flujo principal.
- **Protección y Exclusión Mutua:** La ejecución simultanea de comandos de lectura como ls, y de modificacion como eliminar_archivo(), pueden alterar de forma continua la estructura del sistema.

7. Ejemplos de Uso

El intérprete del simulador de terminal de comandos expone un prompt especializado bajo el formato FiUnamFS \$. A continuación se detallan las pruebas hechas utilizando las imágenes reales del sistema:

1. **Inicialización del Programa y Verificación del Entorno:** Al arrancar el script se monta la imagen binaria mapeando su contenido inicial en la PC.



Figura 1: Ubicación física de los archivos de prueba del sistema desde la computadora.

```
(base) santiagobello@MacBook-Pro-de-Santiago-2 desktop % python FUSE.py /Users/santiagobello/Downloads/fiunamfs.img
Montando sistema de archivos desde: /Users/santiagobello/Downloads/fiunamfs.img
Capa 1 EXITOSA
Capa 2 EXITOSA_Sistema válido
Capa 3 EXITOSA_Version compatible
```

Figura 2: Montaje exitoso, validación de FiUnamFS e inicialización del prompt interactivo.

2. **Mapeo de Directorio mediante el comando ls:** Consulta y renderizado de las estructuras lógicas presentes en el clúster del directorio raíz.

```
¡Bienvenido a FiUnamFS Shell!  
Comandos disponibles: ls, cp, rm, exit  
  
FiUnamFS $ ls  
  
--- ESCANEANDO DIRECTORIO (CON HILOS) ---  
Archivo: IMG_1757.jpg      | Cluster Inicial: 177  
Archivo: logo.png         | Cluster Inicial: 26  
Archivo: mensaje.jpg      | Cluster Inicial: 88  
-----  
Directorio mapeado: 3 archivos encontrados.  
  
--- Contenido del directorio ---  
- IMG_1757.jpg (100191 bytes)  
- logo.png (126423 bytes)  
- mensaje.jpg (180808 bytes)  
-----
```

Figura 3: Ejecución del comando 'ls' desplegando los archivos válidos junto con sus respectivos tamaños.

3. **Transferencia Bilateral de Archivos mediante el comando cp:** Permite el transporte de flujos de bytes de entrada y salida entre el host real y el cluster emulado.

```
FiUnamFS $ cp /Users/santiagobello/Downloads/README.org README.org  
Copiando '/Users/santiagobello/Downloads/README.org' desde tu computadora hacia FiUnamFS...  
  
¡Éxito! 'README.org' copiado a FiUnamFS correctamente.  
  
--- ESCANEANDO DIRECTORIO (CON HILOS) ---  
Archivo: README.org      | Cluster Inicial: 9  
Archivo: IMG_1757.jpg    | Cluster Inicial: 177  
Archivo: logo.png        | Cluster Inicial: 26  
Archivo: mensaje.jpg     | Cluster Inicial: 88  
-----  
Directorio mapeado: 4 archivos encontrados.
```

Figura 4: Copia e inserción de un archivo externo desde el almacenamiento local hacia FiUnamFS.

```

FiUnamFS $ cp logo.png /Users/santiagobello/Desktop
Copiando 'logo.png' hacia tu computadora en '/Users/santiagobello/Desktop'...
¡Éxito! 'logo.png' copiado a tu PC.
FiUnamFS $ cp README.org /Users/santiagobello/Desktop
Copiando 'README.org' hacia tu computadora en '/Users/santiagobello/Desktop'...
¡Éxito! 'README.org' copiado a tu PC.

```

Figura 5: Extracción exitosa de un archivo interno exportándolo directamente al sistema de archivos del computador.

4. **Eliminación Lógica de Registros mediante el comando rm:** Inyección de la firma de invalidación sobrescribiendo las cabeceras binarias mapeadas.

```

FiUnamFS $ rm IMG_1757.jpg
Archivo 'IMG_1757.jpg' eliminado exitosamente.
FiUnamFS $ ls

--- ESCANEANDO DIRECTORIO (CON HILOS) ---
Archivo: README.org      | Cluster Inicial: 9
Archivo: logo.png        | Cluster Inicial: 26
Archivo: mensaje.jpg     | Cluster Inicial: 88
-----
Directorio mapeado: 3 archivos encontrados.

--- Contenido del directorio ---
- README.org (31094 bytes)
- logo.png (126423 bytes)
- mensaje.jpg (180808 bytes)
-----

```

Figura 6: Eliminación y remoción del archivo 'IMG_1757' mediante el borrado lógico de metadatos.

Referencias

- [1] Wolf, Gunnar; Ruiz, Esteban; Bergero, Federico y Meza, Erwin. *Fundamentos de sistemas operativos*. 1ra edición. Ciudad de México: Universidad Nacional Autónoma de México, Instituto de Investigaciones Económicas, Facultad de Ingeniería, 2015. ISBN: 978-607-02-6544-0. Disponible en: <http://sistop.org/>
- [2] Python Software Foundation. *struct — Interpret bytes as packed binary data*. Documentación oficial de Python 3.10. Disponible en: <https://docs.python.org/3/library/struct.html>
- [3] Python Software Foundation. *threading — Thread-based parallelism*. Documentación oficial de Python 3.10. Disponible en: <https://docs.python.org/3/library/threading.html>